LAB 1 (PART B)

SEMESTER 1, SESSION 2026/2027

COURSE CODE	:	**SAIA 3113**
COURSE NAME	:	**AI ENGINEERING**
YEAR/ PROGRAMME	:	**3/ BACHELOR OF ARTIFICIAL INTELLIGENCE**
SECTION	:	**1**
LECTURER'S NAME	:	**PROF. DR. SHAMSUL SAHIBUDDIN**

NAME (MATRIC NUMBER):

1 AMIRAH YASMINE FATHIAH BINTI MOHD REDZUAN (A24AI0097)

2 CHEONG JIAHUI (A24AI0101)

3 SHIRLYN SIEW YU JYE (A24AI0088)

4 TAN QIAN HUI (A24AI0121)

5 WAN NUR HAJAR BINTI WAN AZLAN (A24AI0095)

ENGINEERING

TOPIC 1 LAB — PART B: CONCEPTUAL AI SYSTEM MAP

Group 1–2 | Assigned system: ChatGPT

Lab Activity: Part B - Create a Conceptual AI System Map using Draw.io

This map shows the major components, information flows and potential failure points of ChatGPT, constructed from the behaviour observed in Part A, the concepts introduced in Topic 1 and reasonable engineering assumptions. It is a conceptual approximation, not the actual proprietary architecture.

Step B1 **Open Draw.io and set up (5 minutes)**

The diagram was created in Draw.io on a landscape page titled "ChatGPT — Conceptual AI System Map", using the colour convention below.

Colour	Represents	Components in our map
Green	Data sources and pipeline	Training Data, Web Search
Blue	Model and AI components	GPT Language Model, Ad Selection
Orange	Guardrails and safety	Input Guardrails, Output Guardrails
Purple	Evaluation and monitoring	Feedback Collection, Monitoring & Evaluation
Grey	User interface	Student / General User, Chat Interface

Observation vs assumption:

Components with a **solid border** were directly observed in Part A (the user, the chat interface, and the thumbs up/down and regenerate buttons). Components with a **dashed border** are labelled "Assumed Component" because they are inferred from observed behaviour, for example the Ad Selection component inferred from sponsored ads that matched the conversation topic, and Web Search inferred from ChatGPT's own answer in Test 2.

Step B2 — Draw the conceptual system components (20 minutes)

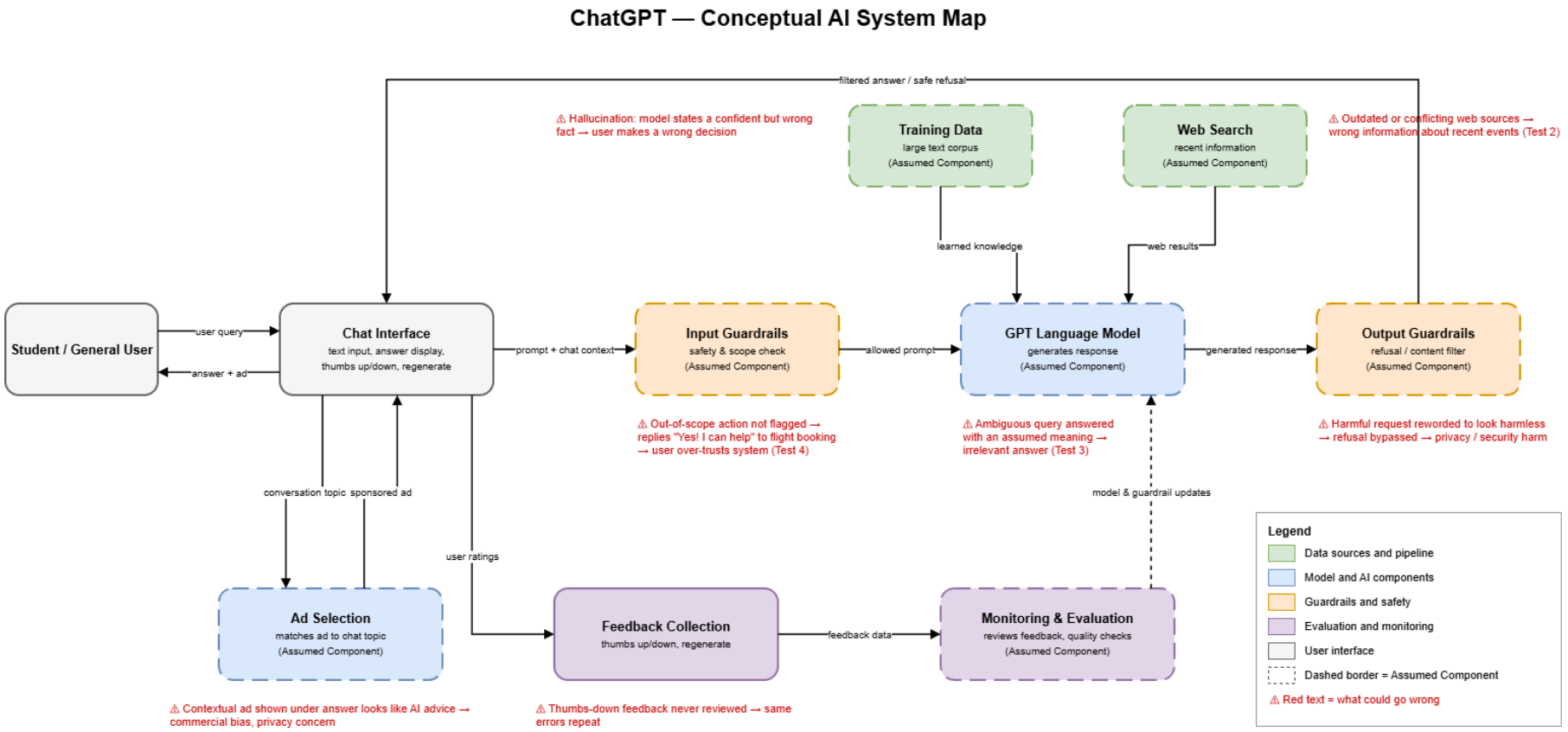


Figure 1: ChatGPT — Conceptual AI System Map (week1_system_map.png)

Step B3 — Annotate with 'What could go wrong?' (15 minutes)

Seven plausible failure points were annotated in red on the map. Table 1 states each failure and its likely user or system impact.

Table 1: Failure points identified on the system map

#	Component	What could go wrong	Likely impact
1	**Input Guardrails**	Out-of-scope action not flagged; ChatGPT replied "Yes! I can help" to a flight booking request (Test 4).	User over-trusts the system and believes it can complete the booking.
2	**GPT Language Model**	Ambiguous query answered with an assumed meaning (Test 3).	User receives an irrelevant answer.
3	**GPT Language Model**	Hallucination: the model states a confident but wrong fact.	User makes a wrong decision based on incorrect information.
4	**Web Search**	Outdated or conflicting web sources used for recent events (Test 2).	User receives wrong information about recent events.
5	**Output Guardrails**	Harmful request reworded to look harmless bypasses the refusal.	Privacy or security harm to account owners or organisations.
6	**Ad Selection**	Contextual ad shown under the answer looks like AI advice.	Commercial bias in what users trust; privacy concern from topic-based ad matching.
7	**Feedback Collection**	Thumbs-down feedback is never reviewed.	The same errors repeat and the system does not improve.

Step B4 — Export and save (5 minutes)

☑ File → Export As → PNG → filename: week1_system_map.png

☑ Saved to computer for committing to GitHub in Part C (docs/week1_system_map.png)

Reflection: Which component or interaction has the most 'what could go wrong?' annotations? What does this suggest about where engineering effort should be concentrated? Connect your answer to the quality dimensions introduced in Topic 1.

The language model and its guardrails have the most "what could go wrong?" annotations, with four of the seven failure points. At the Input Guardrails, an out-of-scope action is not flagged, so ChatGPT replied "Yes! I can help" to a flight booking request (Test 4). At the language model, an ambiguous query was answered with an assumed meaning (Test 3), and the model may also state a confident but incorrect fact. At the Output Guardrails, a harmful request reworded to look harmless could bypass the refusal.

This suggests engineering effort should be concentrated on the generation path, where the response is created and checked, because failures there directly affect the quality dimensions of **accuracy**, **robustness** (handling ambiguous or unexpected inputs) and **safety**. Following the Topic 1 principle of Risk → Control → Evidence, each of these risks needs a control, such as clarification prompts, capability boundary checks and stronger output filters. Each control should also be backed by evidence from systematic testing.

The feedback annotation also shows that **observability** matters: if thumbs-down feedback is never reviewed, these failures cannot be detected or fixed.